



PROJECT (3): CLOUD INFRASTRUCTURE AS A CODE IAC

Team 7



Names	ID's
Ahmed Mohamed Elsayed Tabbash	20P1076
Yaman Abdelhamid Elewa	2002078
Ahmed Abdallah Nofull	22P0255
Ferass Ahmed Mostafa	23P0304

Contents

COMPREHENSIVE TECHNICAL REPORT: HIGH AVAILABILITY CLOUD INFRASTRUCTURE AUTOMATION.....	2
1. Project Overview and Objectives	2
2. Architecture Design Methodology.....	2
3. Parameterization Strategy (params.json).....	4
4. Network Infrastructure Automation (infrastructure.yaml)	5
5. Compute, Load Balancing, and Auto-Scaling Layer	5
6. Database Tier Configuration	6
7. Deployment Execution and Testing.....	7
8. Conclusion	7

COMPREHENSIVE TECHNICAL REPORT: HIGH AVAILABILITY CLOUD INFRASTRUCTURE AUTOMATION

1. Project Overview and Objectives

The primary objective of this project was to design, build, and automate a highly available, enterprise-ready cloud infrastructure using AWS CloudFormation. Moving away from manual infrastructure configuration, this implementation utilized Infrastructure as Code (IaC) to benefit from conventional software development practices. By treating infrastructure as software, the project achieved improved deployment speed, minimized human configuration errors, and eliminated configuration drift while ensuring consistent security strategies.

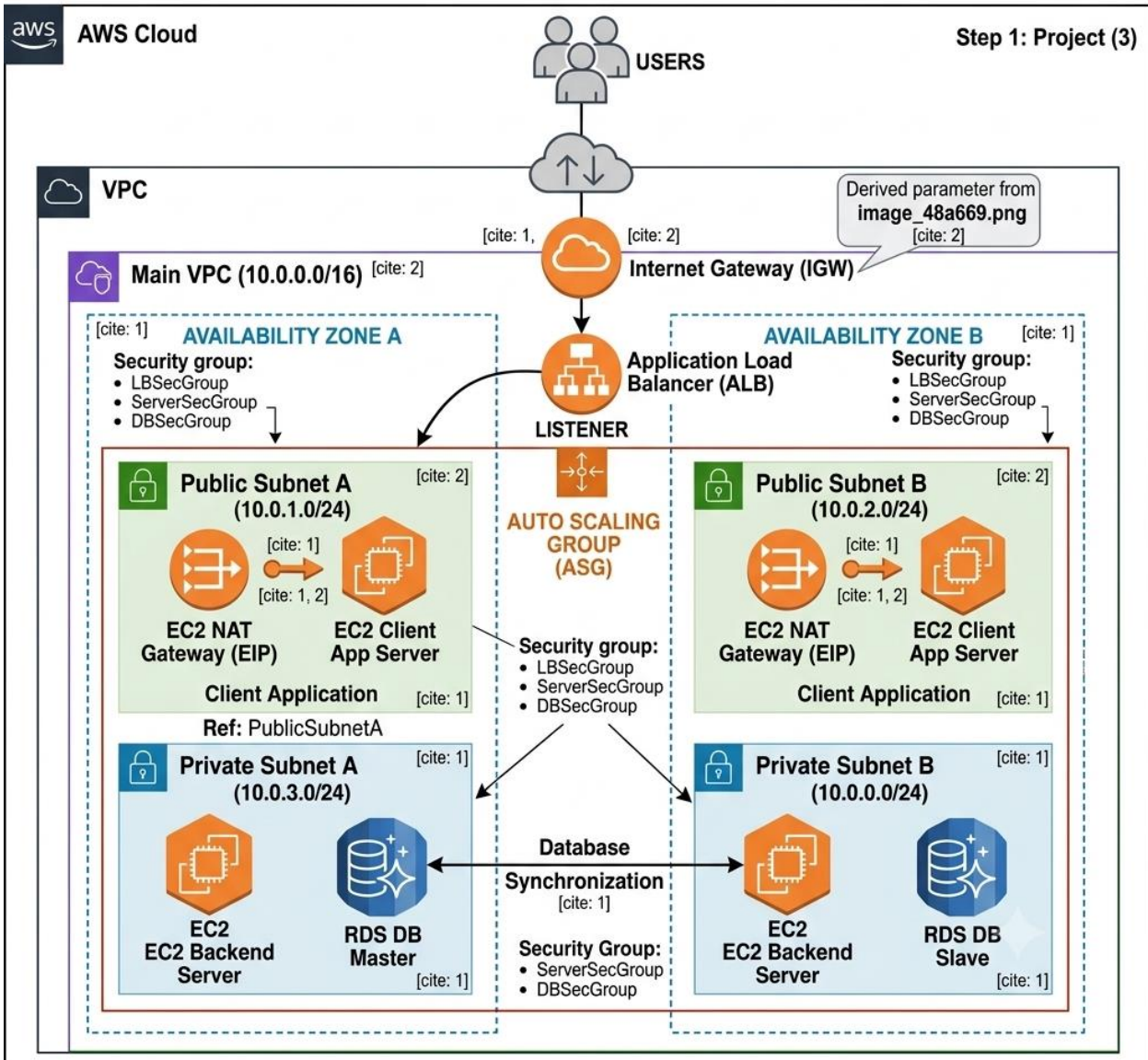
The target architecture required the automated creation of a Virtual Private Cloud (VPC), Public and Private subnets spanning multiple Availability Zones (AZs), NAT Gateways, an Internet Gateway (IGW), Elastic Compute Cloud (EC2) instances, an Application Load Balancer (ALB), and a synchronized database.

2. Architecture Design Methodology

Before writing the IaC scripts, the network architecture was visually modeled using the Lucid Chart tool. To ensure fault tolerance and high availability, the architecture was explicitly divided across two distinct Availability Zones: Zone A and Zone B.

- **Public Tier:** Public Subnet A and Public Subnet B were provisioned to host outward-facing resources, specifically the Application Load Balancer and NAT Gateways. Client applications interface with this tier first.
- **Private Tier:** Private Subnet A and Private Subnet B were designed to secure backend servers and databases, completely isolating them from direct internet access.

- **Database Synchronization:** An Amazon Relational Database Service (RDS) deployment was mapped out, featuring an RDS DB Master in Private Subnet A and a synchronized RDS DB Slave in Private Subnet B.



3. Parameterization Strategy (`params.json`)

To facilitate reusability and maintainability, the infrastructure configuration was separated from the declarative code using a JSON parameter file. This file acts as the single source of truth for the network's Classless Inter-Domain Routing (CIDR) blocks.

Parameter Key	Assigned Value	Description
EnvironmentName	HA-Project	The global prefix used for tagging and identifying stack resources.
VpcNetCIDR	10.0.0.0/16	The overarching IP range for the entire Virtual Private Cloud.
PubSubnet01CIDR	10.0.1.0/24	IP range allocated to Public Subnet A in Availability Zone 1.
PubSubnet02CIDR	10.0.2.0/24	IP range allocated to Public Subnet B in Availability Zone 2.
PrivSubnet01CIDR	10.0.3.0/24	IP range allocated to Private Subnet A in Availability Zone 1.
PrivSubnet02CIDR	10.0.0.0/24	IP range allocated to Private Subnet B in Availability Zone 2.

4. Network Infrastructure Automation (infrastructure.yaml)

The foundational network layer was automated using AWS CloudFormation in YAML format. The following components were programmatically provisioned:

- **Virtual Private Cloud (VPC):** Configured with DNS support and hostnames enabled to act as the secure container for all subsequent resources.
- **Subnet Allocation:** Four subnets were created utilizing the `!Select` and `!GetAZs` intrinsic functions to ensure they were evenly distributed across the region's available zones. Public IP mapping on launch was explicitly disabled for private subnets to enforce security compliance.
- **Internet Gateway (IGW):** Attached directly to the VPC to act as the primary node for external internet communication.
- **NAT Gateways & Elastic IPs (EIP):** Two NAT Gateways were provisioned—one in each public subnet—and assigned dedicated Elastic IPs. This allows resources in the private subnets to securely download updates without exposing their private IP addresses.
- **Route Tables:** A public route table was created with a `0.0.0.0/0` route pointing to the IGW, shared by both public subnets. Two distinct private route tables were created, each directing outbound traffic (`0.0.0.0/0`) to their respective AZ's NAT Gateway.

5. Compute, Load Balancing, and Auto-Scaling Layer

To handle variable application loads and ensure business continuity, the compute layer was dynamically configured.

- **Security Groups:** Deployed as virtual firewalls. The `LBSecGroup` allows inbound HTTP (Port 80) traffic from the public internet. The `ServerSecGroup` restricts backend EC2 access strictly to traffic originating from the Load Balancer.
- **Application Load Balancer (ALB):** Provisioned across both public subnets to distribute incoming client traffic evenly. A Target Group and Listener were configured to perform continuous health checks on the backend instances.

- **Launch Template & User Data:** Replaced legacy configurations with a modern EC2 Launch Template. An Amazon Linux AMI was selected, and a bash script was injected via the `UserData` property. This script fully automates the installation of the Apache web server (`httpd`), PHP, and MySQL drivers upon instance initialization.
- **Auto-Scaling Group (ASG):** Configured with a minimum capacity of 2 and a maximum of 3 instances. The ASG spans both public subnets and is directly bound to the ALB's Target Group to ensure traffic is only routed to healthy nodes.

6. Database Tier Configuration

The enterprise data storage requirement was fulfilled utilizing a fully managed database service.

- **Resource Specifications:** An AWS RDS instance was provisioned utilizing the MySQL engine on a `db.t3.micro` instance class with 20GB of encrypted GP2 storage.
- **High Availability (Multi-AZ):** To meet the master-slave synchronization requirement, the `MultiAZ` parameter was set to `true`. This instructs AWS to automatically provision a synchronous standby replica in the secondary Availability Zone, ensuring immediate failover in the event of hardware degradation.
- **Security Integration:** A `DBSubnetGroup` was established to lock the database instances strictly within the private subnets. Furthermore, database credentials were omitted from the plaintext YAML file; instead, AWS Secrets Manager was utilized (`ManageMasterUserPassword: true`) to dynamically generate and secure the master user password.

7. Deployment Execution and Testing

The entire stack was validated and deployed via the AWS Command Line Interface (CLI) to eliminate manual graphical user interface (GUI) navigation.

- **Execution Command:** The deployment was triggered using the standard `create-stack` command, passing both the template body and the parameter JSON file:

```
aws cloudformation create-stack --stack-name HA-Project-Stack --region us-east-1 --template-body file://infrastructure.yaml --parameters file://params.json
```

- **Linter Compliance:** Throughout the development lifecycle, the code was strictly optimized to clear all CloudFormation linter warnings, including migrating from deprecated Launch Configurations to modern Launch Templates and defining explicit stateful deletion policies (Retain) to prevent accidental data loss on the RDS resource.

8. Conclusion

The project successfully demonstrated the power and efficiency of Infrastructure as Code. By writing declarative YAML and JSON scripts, an entire enterprise-grade, highly available, and secure cloud environment was provisioned automatically. The resulting infrastructure strictly adheres to the architectural requirements, distributing load across multiple Availability Zones while keeping critical compute and database resources securely isolated within private networks.